

No binary $[[14, 3, d \geq 5]]$ quantum stabilizer code exists

A signed-shadow proof and exhaustive additive-code classification

Brandon Li

Independent researcher

brandon.li.gn@gmail.com

Abstract

We prove that no binary qubit stabilizer code with parameters $[[14, 3, d \geq 5]]$ exists, settling the remaining distance gap for stabilizer codes encoding three qubits into fourteen. The proof first applies a signed affine-shadow inequality and an integrality congruence to restrict low-weight stabilizer words. Split-shadow Farkas certificates then exclude weight-two words and every configuration with three weight-four stabilizer words, leaving a unique weight-four stabilizer word.

That word yields a ten-coordinate binary-additive code of size 1,024, minimum distance at least five, and symplectic hull dimension four. By exhaustive lengthening, we reconstruct all 37 equivalence classes of length-ten binary-additive codes of size 1,024 and minimum distance at least five. Of these, 25 have symplectic hull dimension different from four, and each of the 12 with hull dimension four violates a necessary coset-capacity bound. Together with the known $[[14, 3, 4]]$ construction, this proves $d_{\max}(14, 3) = 4$ for binary stabilizer codes.

Keywords: quantum error correction; stabilizer codes; additive codes; shadow enumerators; linear programming bounds; computer-assisted proof

Mathematics Subject Classification (2020): 81P73, 94B65, 94B05

1 Introduction and main result

Theorem 1 (Nonexistence). *There is no binary qubit stabilizer code with parameters $[[14, 3, d \geq 5]]$.*

Corollary. *The maximum distance of a binary qubit stabilizer code with $n = 14$ and $k = 3$ is $d_{\max}(14, 3) = 4$.*

Proof. Theorem 1 gives the upper bound, and the known $[[14, 3, 4]]$ construction gives the matching lower bound [8]. $\square$

Grassl’s quantum-code tables [8] record a $[[14, 3, 4]]$ construction and an upper bound of five for the distance of a binary $[[14, 3]]$ stabilizer code. Ball, Centelles, and Huber [4] identified $[[14, 3, 5]]$ in Research Problem 1 as the smallest parameter set for which the existence of a qubit stabilizer code was then unknown. Theorem 1 resolves that question in the negative. The nonexistence proof for $[[13, 5, 4]]$ by Bierbrauer, Fears, Marcugini, and Pambianco [5] provides a related precedent. Our argument combines signed weight enumerators [2, 3] with the classification of binary-additive codes; it does not assume $\text{GF}(4)$ -linearity.

The proof has three steps. First, signed-shadow identities restrict the possible low-weight stabilizer words. Second, linear infeasibility certificates eliminate the remaining low-weight configurations. Third,

a symplectic reduction leads to an exhaustive classification of ten-site additive codes and a final coset-counting contradiction. The computational verification is summarized in Section 9.

Here, *physical* refers to weight or support on the original qubits, as distinct from a coordinate of an auxiliary relaxation. The result applies to both pure and degenerate binary stabilizer codes, but not to arbitrary non-stabilizer quantum codes.

2 Pauli space and notation

Let

$$V_n = \mathbb{F}_2^{2n} = \{(x \mid z) : x, z \in \mathbb{F}_2^n\},$$

encode Pauli labels modulo phase. The symplectic form is

$$\langle (x \mid z), (x' \mid z') \rangle = x \cdot z' + z \cdot x' \in \mathbb{F}_2.$$

The Pauli weight is

$$\text{wt}(x \mid z) = \#\{i : (x_i, z_i) \neq (0, 0)\}.$$

A binary stabilizer code with parameters $\llbracket 14, 3, d \rrbracket$ is an isotropic subspace $S \leq V_{14}$ of binary dimension $14 - 3 = 11$. Its centralizer is $S^\perp$, and its distance is

$$d(S) = \min\{\text{wt}(v) : v \in S^\perp \setminus S\}.$$

We call each element of S a stabilizer word. A stabilizer generator is an element of a chosen basis of S ; weight enumerators count all stabilizer words, not only basis generators.

For any nonzero binary subspace C of a Pauli space, write

$$d(C) = \min\{\text{wt}(c) : c \in C \setminus \{0\}\}.$$

We write $d_{\max}(n, k)$ for the greatest minimum distance $d(S)$ among binary stabilizer codes with fixed n and k ; this should not be confused with the maximum of the minimum weights of distinct logical-Pauli cosets within one fixed code.

We assume for contradiction that $d(S) \geq 5$. A code is *pure to distance d* if no nonzero stabilizer word has weight below d ; otherwise it is degenerate (or impure). In particular, the distance formula does not exclude low-weight stabilizer words. If a vector of weight at most four commutes with S , it must itself be in S .

For a binary subspace C of a Pauli space, its symplectic hull is $\text{Hull}(C) = C \cap C^\perp$. When a block of coordinates is specified, $H_{a,b}$ below counts stabilizer words of split weight (a, b) ; it is an enumerator, not a subspace. The hull in Section 6 is denoted J to keep the two notions distinct.

3 The signed affine shadow, including split weights

For one qubit define

$$q(x, z) = x + z + xz \in \mathbb{F}_2.$$

This is one exactly when (x, z) is nonzero, so on V_n it satisfies

$$\begin{aligned} q(v) &= \text{wt}(v) \pmod{2}, \\ q(v+w) &= q(v) + q(w) + \langle v, w \rangle. \end{aligned}$$

Because S is isotropic, $q|_S$ is linear. Nondegeneracy of the symplectic form gives a vector v_0 such that

$$\langle v_0, s \rangle = q(s) \quad \forall s \in S.$$

Character orthogonality gives the indicator identity

$$1_{v_0+S^\perp}(v) = 2^{-11} \sum_{s \in S} (-1)^{q(s) + \langle v, s \rangle}.$$

Thus the signed transform counts weights in the affine coset $v_0 + S^\perp$, so every split cell is a nonnegative integer.

Fix a partition of the fourteen qubits into blocks of lengths m and f , with $m + f = 14$, and let $H_{i,j}$ count stabilizer words with block weights (i, j) . Define the quaternary Krawtchouk coefficients

$$K_a^m(i) = [u^a](1 + 3u)^{m-i}(1 - u)^i.$$

If $C_{a,b}$ counts $S^\perp$ and $\text{Sh}_{a,b}$ counts the affine shadow, then

$$\begin{aligned} 2048C_{a,b} &= \sum_{i,j} H_{i,j} K_a^m(i) K_b^f(j), \\ 2048\text{Sh}_{a,b} &= \sum_{i,j} (-1)^{i+j} H_{i,j} K_a^m(i) K_b^f(j). \end{aligned}$$

For use in the split-weight systems, write $T_{a,b} = 2048C_{a,b}$ and $\text{SH}_{a,b} = 2048\text{Sh}_{a,b}$; these are the integer transform numerators in the displayed formulas.

Both $C_{a,b}$ and $\text{Sh}_{a,b}$ are nonnegative integers, and $C_{a,b} \geq H_{a,b}$ because $S \leq S^\perp$. If $1 \leq a + b \leq 4$, then

$$C_{a,b} = H_{a,b},$$

because every centralizer word in that weight range must be a stabilizer word when $d(S) \geq 5$. This equality is valid even when S is impure.

4 Global low-weight restrictions and parity

Write A_j for the number of stabilizer words of total physical weight j , and put

$$\begin{aligned} T_j &= \sum_w A_w K_j^{14}(w), \\ U_j &= \sum_w (-1)^w A_w K_j^{14}(w). \end{aligned}$$

Write $\text{Sh}_j = \sum_{a+b=j} \text{Sh}_{a,b}$ for the total-weight count in the affine shadow. The quantities $T_j/2048$ and $U_j/2048$ are the actual total-weight counts in $S^\perp$ and in the affine shadow. For the fixed partition of Section 3, coefficient convolution gives

$$\begin{aligned}\sum_{a+b=j} K_a^m(i) K_b^f(\ell) &= [t^j](1+3t)^{m-i}(1-t)^i(1+3t)^{f-\ell}(1-t)^\ell \\ &= K_j^{14}(i+\ell).\end{aligned}$$

Summing the split-shadow transform of Section 3 over $a+b=j$ therefore gives

$$U_j = \sum_{a+b=j} \text{SH}_{a,b} = 2048 \text{Sh}_j.$$

Hence $U_j \geq 0$, even without purity. Also $A_0 = 1$, $\sum_j A_j = 2048$, and $T_j = 2048A_j$ for $j = 1, 2, 3, 4$.

Lemma 2 (Low-weight restriction). *If $d(S) \geq 5$, then $A_1 = A_3 = 0$, $A_2 \leq 2$, and $A_4 \leq 3$.*

Proof. Expanding the Krawtchouk transforms in T_j and U_j gives the coefficientwise identity

$$\begin{aligned}989184A_4 &= -296U_2 - 88U_4 - 9584640A_0 + 6309 \sum_j A_j \\ &\quad - 11685888A_1 - 1345536A_2 - 4681728A_3 \\ &\quad + 1914(T_1 - 2048A_1) + 1318(T_2 - 2048A_2) \\ &\quad + 279(T_3 - 2048A_3) + 161(T_4 - 2048A_4) \\ &\quad - 1081344A_5 - 254976A_6 - 15360A_7 - 44032A_{14}.\end{aligned}$$

Now $A_0 = 1$, $\sum_j A_j = 2048$, and $T_j = 2048A_j$ for $1 \leq j \leq 4$. Since $U_2, U_4, A_5, A_6, A_7, A_{14} \geq 0$, the identity implies

$$11685888A_1 + 1345536A_2 + 4681728A_3 + 989184A_4 \leq 3336192.$$

Since the A_j are nonnegative integers, these coefficients imply

$$A_1 = A_3 = 0, \quad A_2 \leq 2, \quad A_4 \leq 3.$$

□

Remark. Section 5.3 excludes the remaining cases $A_2 \in \{1, 2\}$.

4.1 Integral affine-shadow parity

The inequality alone does not determine whether A_4 is zero. An integrality congruence supplies the missing condition. We choose the coefficients below so that expanding the MacWilliams and shadow equations leaves, modulo 4096, only the terms in A_3 and A_4 . Let

$$\begin{aligned}E_0 &= \sum_w A_w = 2048, \\ E_j &= T_j - 2048A_j, \quad j = 1, \dots, 4, \\ F_j &= U_j - 2048\text{Sh}_j, \quad j = 0, \dots, 3.\end{aligned}$$

Take the integer combination

$$L = E_0 - 2E_1 - 6E_2 - 17E_3 - 95E_4 \\ - 20F_0 - 24F_1 + 1108F_2 + 246F_3.$$

The equations give $L = 2048$. Writing c_j for the coefficient of A_j , direct expansion gives

$$(c_0, \dots, c_{14}) = (-4550656, -7348224, -1052672, -2623488, 321536, \\ -634880, 221184, -28672, 24576, 24576, \\ -45056, 0, 40960, -16384, -77824).$$

Modulo 4096 every coefficient vanishes except those of A_3 and A_4 , each congruent to 2048. The four shadow-variable coefficients are (40960, 49152, -2269184, -503808), all divisible by 4096. Since $L = 2048$, reduction modulo 4096 gives

$$A_3 + A_4 \equiv 1 \pmod{2}.$$

Together with $A_3 = 0$ and $0 \leq A_4 \leq 3$, this gives

$$A_4 \in \{1, 3\}.$$

5 Excluding low-weight stabilizer configurations

For each specified subgroup, we derive a relaxation satisfied by every hypothetical code containing it. A Farkas certificate excludes the remaining low-weight configurations when the relaxation is infeasible. The exclusions are used in the following order, and no step depends on a later one: Lemma 2 and the parity congruence (Section 4) $\rightarrow$ Section 5.3 ($A_2 = 0$, using only Lemma 2) $\rightarrow$ Section 5.4 (pairs of weight-four words) $\rightarrow$ Section 5.5 (support geometry) $\rightarrow$ Sections 5.6–5.7 (the $A_4 = 3$ geometries, which also use $A_2 = 0$) $\rightarrow$ Lemma 3.

5.1 A necessary coset-counting relaxation

Let $R \leq S$ be an isotropic subgroup supported on m of the original qubits, with the remaining $f = 14 - m$ sites forming a suffix. Every element of S commutes with R , so its prefix lies in $R^\perp$. Also, S is a union of complete R -cosets.

For each complete coset Q of R in $R^\perp$, let

$$p(Q) = (p_0, \dots, p_m), \\ p_a = \#\{q \in Q : \text{wt}_{\text{pref}}(q) = a\}, \quad 0 \leq a \leq m.$$

Here wt_{pref} is Pauli weight on the original prefix sites. Let $y_{p,b}$ be the number of complete R -cosets selected by S , aggregated over all suffix Pauli labels of weight b , and grouped only by the prefix pattern p . Then $y_{p,b} \geq 0$ and

$$H_{a,b} = \sum_p p_a y_{p,b}.$$

The model requires only $R \leq S$; it does not assume that R is all of $S \cap V_{\text{pref}}$. If S contains further prefix-supported words, they are still counted correctly, because S is a union of complete R -cosets. This is a necessary model for a real code. Appendix A defines every row and column of each such system

exactly, and lists the coset patterns and certificate data. It deliberately forgets the actual prefix coset label, the suffix Pauli label, and all global linear compatibility among different suffixes. It therefore enlarges the feasible set. A contradiction in this relaxation excludes a physical code; feasibility would prove nothing.

For every such model we use the following row families:

- $H_{0,0} = 1$ and $\sum_{a,b} H_{a,b} = 2048$;
- physical coset equalities or inequalities forced by the selected subgroup;
- the low-weight distance equations $T_{a,b} - 2048H_{a,b} = 0$ for $1 \leq a + b \leq 4$;
- centralizer inclusion $2048H_{a,b} - T_{a,b} \leq 0$ in every split cell;
- centralizer nonnegativity $-T_{a,b} \leq 0$ in every split cell;
- signed affine-shadow nonnegativity $-\text{SH}_{a,b} \leq 0$ in every split cell; and
- $y \geq 0$.

The transforms $T_{a,b}$ and $\text{SH}_{a,b}$ use Pauli weights on the original prefix and suffix sites. The coset-capacity bound of Section 6.2 is not used in these exclusions.

5.2 Exact Farkas sign convention

After choosing the variables for a particular relaxation and scaling by 2048, write the necessary system as

$$Mz = \text{rhs}, \quad Gz \leq 0, \quad z \geq 0.$$

A Farkas certificate consists of nonnegative λ , free-sign ν , and a coefficient vector c satisfying

$$\begin{aligned} \lambda &\geq 0, \\ c &= \lambda G + \nu M, \quad c \geq 0, \\ \nu \cdot \text{rhs} &< 0. \end{aligned}$$

The inequality $c \geq 0$ is entrywise.

For a putative feasible z ,

$$\begin{aligned} 0 \leq c \cdot z &= \lambda \cdot (Gz) + \nu \cdot (Mz) \\ &\leq \nu \cdot \text{rhs} < 0, \end{aligned}$$

which is impossible. We check each certificate using exact integer arithmetic. The constraints are derived from the stabilizer problem before applying Farkas' lemma; the certificate shows that the resulting linear system is infeasible.

5.3 Excluding weight-two stabilizers

Suppose $p = Z_0Z_1$ belongs to S . A one-qubit Clifford acts transitively on the three nonidentity Pauli labels, so local Clifford conjugations on sites 0 and 1 send any weight-two stabilizer word to this representative while preserving weight and the code parameters. In particular, Hadamards send X_0X_1 to Z_0Z_1 . The eight two-site Pauli labels commuting with p split into four complete cosets of $\langle p \rangle$ with two-site pattern

$$(1, 0, 1) : 1, \quad (0, 2, 0) : 1, \quad (0, 0, 2) : 2.$$

The integers after the colons are the coset multiplicities.

For every fixed suffix Pauli label, complete-coset closure therefore gives

$$H_{2,b} \geq H_{0,b}, \quad b = 0, \dots, 12.$$

The 39 variables are the split counts $H_{a,b}$ of stabilizer words with prefix weight $a \in \{0, 1, 2\}$ on sites 0 and 1 and suffix weight $b \in \{0, \dots, 12\}$ on the remaining sites.

Every hypothetical code satisfies the following constraints:

- $H_{0,0} = 1$, $\sum_{a,b} H_{a,b} = 2048$, and $H_{a,b} \geq 0$;
- the 11 equalities $T_{a,b} = 2048H_{a,b}$ for $1 \leq a + b \leq 4$;
- the 13 coset inequalities $H_{2,b} \geq H_{0,b}$; and
- in each of the 39 split cells, centralizer inclusion $T_{a,b} \geq 2048H_{a,b}$, centralizer nonnegativity $T_{a,b} \geq 0$, and affine-shadow nonnegativity $\text{SH}_{a,b} \geq 0$.

Together with the two normalization equalities, these give 13 equality rows. The inequalities give 130 rows, in addition to the 39 nonnegativity conditions on the variables. This relaxation omits the further coset-pattern relations in Section 5.1.

The Farkas certificate has 13 nonnegative inequality multipliers and 10 nonzero equality multipliers. The residual vector c has 17 positive and 22 zero entries and satisfies the 39 column identities $c = \lambda G + \nu M$. Its normalization is

$$\nu \cdot \text{rhs} = -2,571,108,352.$$

Thus $A_2 = 0$ for every hypothetical code.

5.4 Excluding overlapping low-weight stabilizer words

The support argument in Section 5.5 uses the following exclusions. Each subgroup acts on the original qubits, and its normalizer and cosets are enumerated before the linear relaxation is formed.

5.4.1 A weight-three subgroup

For $R = \langle \text{ZZZ} \rangle$ on three original sites, the 16 normalizer cosets have prefix patterns

$$\begin{aligned} (0, 0, 1, 1) &: 12, \\ (0, 1, 1, 0) &: 3, \\ (1, 0, 0, 1) &: 1. \end{aligned}$$

As above, the integer after each colon is the number of cosets with that prefix-weight pattern.

The Farkas certificate satisfies the sign conditions of Section 5.2 on all 36 columns and has $\nu \cdot \text{rhs} = -22,528$. It also excludes a weight-three stabilizer word without invoking Lemma 2.

5.4.2 The four-site Bell subgroup

For $R = \langle \text{XXXX}, \text{ZZZZ} \rangle$ on four original sites, the 16 complete cosets have patterns

$$\begin{aligned} (1, 0, 0, 0, 3) &: 1, \\ (0, 0, 2, 0, 2) &: 9, \\ (0, 0, 0, 4, 0) &: 6. \end{aligned}$$

The integers after the colons are the coset multiplicities.

For every suffix weight b ,

$$\begin{aligned} H_{1,b} &= 0, \\ H_{4,b} &= H_{2,b} + 3H_{0,b}. \end{aligned}$$

The Farkas certificate satisfies the sign conditions of Section 5.2 on all 55 columns and has

$$\nu \cdot \text{rhs} = -320,017,816,092,672.$$

This excludes a pair of weight-four Bell stabilizer words with the same support.

5.4.3 The five-site overlap subgroup

If two commuting weight-four words overlap in three sites and differ on two of those sites, choose a local Clifford operation independently at each site. At each of the two sites where the letters differ, map the ordered pair of letters to (X, Z) . At the third common site, and at each site used by only one word, map the letter to Z . After a site permutation this gives

$$\begin{aligned} p &= X_0 X_1 Z_2 Z_3 I_4, \\ q &= Z_0 Z_1 Z_2 I_3 Z_4. \end{aligned}$$

All three nonzero words in $\langle p, q \rangle$ have weight four on five sites. The normalizer has 64 four-element cosets and nine prefix-weight patterns. The Farkas certificate satisfies the sign conditions of Section 5.2 on all 90 columns and has

$$\nu \cdot \text{rhs} = -52,501,014,528.$$

5.4.4 The two six-site overlap subgroups

For the same-letter two-site overlap, use

$$R = \langle \text{ZZZZII}, \text{ZZIIZZ} \rangle.$$

The six-site normalizer has 256 four-element cosets and 13 prefix patterns. The corresponding 117-variable system has a Farkas certificate satisfying the sign conditions of Section 5.2 on all 117 columns, with $\nu \cdot \text{rhs} = -8,820,736$.

For the crossed two-site overlap, use

$$R = \langle \text{ZZZZII}, \text{XXIIXX} \rangle.$$

The nonzero subgroup words have weights 4, 4, 6. Here the normalizer has 14 prefix patterns; the resulting 126-variable system has a Farkas certificate satisfying the sign conditions of Section 5.2 on all 126 columns, with $\nu \cdot \text{rhs} = -22,212,608$. We check both six-site exclusions separately, enumerating the complete cosets in each case.

5.5 Support geometry of weight-four stabilizer words

Let p, q be distinct commuting weight-four stabilizer words. Let t be the number of common support sites and h the number of common sites on which their nonidentity Pauli letters differ. Commutation makes h even, and

$$\text{wt}(p + q) = 8 - 2t + h.$$

The preceding subgroup exclusions remove every case with $t \geq 2$, as shown in Table 1:

Table 1: Subgroup exclusions covering every pair of distinct commuting weight-four stabilizer words that overlap in $t \geq 2$ sites, where h counts the common sites carrying different Pauli letters.

t	h	obstruction
4	2	weight-two stabilizer
4	4	four-site Bell subgroup
3	0	weight-two stabilizer
3	2	five-site overlap subgroup
2	0	six-site same-letter triangle subgroup
2	2	six-site crossed-overlap subgroup

The omitted case $t = 4, h = 0$ would give the same Pauli word twice, not two distinct stabilizer words. Thus two distinct weight-four stabilizer words meet in at most one qubit. If they meet in one, commutation forces the same local Pauli letter there. Hence every qubit carries at most one nonidentity letter across all weight-four stabilizer words, and a single local Clifford operation at each qubit sends that letter to Z . Local Clifford transformations therefore map all weight-four stabilizer words to Z -only words simultaneously. This does not imply that the full stabilizer is CSS.

If $A_4 = 3$, the three weight-four stabilizer words are linearly independent: the sum of any two has weight six or eight, not four. Their support sets are pairwise disjoint or meet in one site. Up to site permutation and reordering, there are exactly five support patterns:

Table 2: The five support patterns for three weight-four stabilizer words when $A_4 = 3$, up to site permutation and reordering. Only the first row has pairwise disjoint supports.

type	word supports	union	pair-product weights	triple-product weight
0 intersections	$\{0, 1, 2, 3\}; \{4, 5, 6, 7\}; \{8, 9, 10, 11\}$	12	8, 8, 8	12
1 intersection	$\{0, 1, 2, 3\}; \{3, 4, 5, 6\}; \{7, 8, 9, 10\}$	11	6, 8, 8	10
2 intersections	$\{0, 1, 2, 3\}; \{3, 4, 5, 6\}; \{6, 7, 8, 9\}$	10	6, 8, 6	8
3 distinct intersections	$\{0, 1, 2, 3\}; \{3, 4, 5, 6\}; \{2, 4, 7, 8\}$	9	6, 6, 6	6
one common site	$\{0, 1, 2, 3\}; \{3, 4, 5, 6\}; \{3, 7, 8, 9\}$	10	6, 6, 6	10

To see that the list is complete, record which of the three pairs of supports intersect. There can be zero, one, two, or three intersecting pairs. When exactly two pairs intersect, their intersection sites must be distinct: if both were the same, that site would lie in all three supports and the third pair would intersect too. When all three pairs intersect, either their three intersection sites differ or all three supports share one site. These are precisely the five cases above.

5.6 The four overlapping rank-three geometries

For each of the four non-disjoint rows of Table 2, let R be the rank-three subgroup generated by the three displayed Z-only words. Its cosets in the normalizer are grouped by prefix-weight histograms. The relaxations are summarized in Table 3:

In each branch, the system includes the global equations

$$\sum_{a+b=j} H_{a,b} = 0, \quad j = 1, 2, 3,$$

$$\sum_{a+b=4} H_{a,b} = 3.$$

The first three follow from the low-weight restrictions and the weight-two exclusion; the last is the branch assumption $A_4 = 3$. They are necessary conditions on the hypothetical code, not consequences of subgroup containment alone.

Table 3: Farkas certificates excluding the four non-disjoint geometries of Table 2.

geometry	sites	patterns	variables	positive multipliers	$\nu \cdot \text{rhs}$
three distinct intersections	9+5	40	240	12	−86,470,656
one common site	10+4	27	135	13	−9,031,680
two intersections	10+4	60	300	14	−20,484,096
one intersection	11+3	74	296	14	−61,440

For each row, the Farkas certificate has nonnegative inequality multipliers and nonnegative residuals in every column, with the negative normalization shown. Farkas’ lemma then excludes the system. These systems also impose the global equations

$$A_1 = A_2 = A_3 = 0, \quad A_4 = 3.$$

5.7 The disjoint $4 + 4 + 4 + 2$ geometry

The only possible $A_4 = 3$ geometry left by Section 5.6 would be

$$R = \langle \text{ZZZZIIIIIIIIII}, \text{IIIIZZZZIIIIII}, \text{IIIIIIIIZZZZII} \rangle.$$

Each four-site block has 64 cosets of $\langle \text{ZZZZ} \rangle$ in its even-X-parity normalizer. The six weight patterns of Table 4 have multiplicities totaling $1 + 4 + 3 + 24 + 24 + 8 = 64$:

Table 4: Prefix-weight patterns of the 64 cosets of $\langle \text{ZZZZ} \rangle$ in one four-site block.

pattern $(p_0, p_1, p_2, p_3, p_4)$	multiplicity
(1, 0, 0, 0, 1)	1
(0, 1, 0, 1, 0)	4
(0, 0, 2, 0, 0)	3
(0, 0, 1, 0, 1)	24
(0, 0, 0, 2, 0)	24
(0, 0, 0, 0, 2)	8

For each of the three blocks and the two untouched sites, let $y_{p,q,r,b}$ count complete R -cosets grouped by those literal patterns and suffix weight b . There are $6 \cdot 6 \cdot 6 \cdot 3 = 648$ nonnegative variables. The grouping forgets the actual coset labels, suffix letters, and global linearity, so it is a relaxation.

The four-block split enumerator has $5^3 \cdot 3 = 375$ cells, indexed by the three four-site block weights and the weight on the two untouched sites, and the system has 70 equalities. A Farkas certificate using the signed-shadow nonnegativity constraints gives

$$\begin{aligned}\lambda &\geq 0, \\ c &= \lambda G + \nu M \geq 0, \\ \nu \cdot \text{rhs} &= -125,829,120.\end{aligned}$$

The equality rows include the same global weight equations used in Section 5.6: $A_1 = A_2 = A_3 = 0$ and the branch assumption $A_4 = 3$. They do not follow from containment of the disjoint subgroup alone. All 648 residual coefficients are nonnegative (403 positive and 245 zero), so the Farkas certificate excludes this final geometry as well.

Lemma 3 (Unique weight-four word). *Every binary $[[14, 3, d \geq 5]]$ stabilizer code has exactly one stabilizer word of weight four.*

Proof. The parity restriction gives $A_4 \in \{1, 3\}$, and Sections 5.3–5.7 exclude $A_4 = 3$. Hence $A_4 = 1$. $\square$

6 Reduction to a ten-site additive code

Lemma 4 (Ten-site reduction). *Suppose $S \leq V_{14}$ is isotropic of dimension 11, has $d(S) \geq 5$, and its only nonzero stabilizer word of physical weight at most four is a word p of weight four. Equivalently, $A_1 = A_2 = A_3 = 0$ and $A_4 = 1$. Then there exists a binary subspace $C \leq V_{10}$ of dimension 10, minimum Pauli weight at least five, and $\dim(C \cap C^\perp) = 4$.*

Proof. Let p be the unique weight-four stabilizer word. After local Clifford conjugation and a site permutation, assume $p = Z_0 Z_1 Z_2 Z_3$ on the first four original qubits. Write V_4 for their Pauli space and W_{10} for the remaining ten-qubit Pauli space, so $V_{14} = V_4 \oplus W_{10}$.

Define the physical six-dimensional quotient

$$E = (p^\perp \cap V_4) / \langle p \rangle.$$

The quotient is nondegenerate symplectic: $p^\perp \cap V_4$ has binary dimension seven, and quotienting by its one-dimensional radical $\langle p \rangle$ leaves dimension six.

By the hypothesis, $S \cap V_4 = \langle p \rangle$, so projection of $S / \langle p \rangle$ to W_{10} is injective. Let its image be a binary ten-space $P \leq W_{10}$. Then there is a linear map $T : P \rightarrow E$ such that

$$\begin{aligned}S / \langle p \rangle &= \{T(x) + x : x \in P\}, \\ \langle x, y \rangle_W &= \langle T(x), T(y) \rangle_E.\end{aligned}$$

6.1 Distance forces surjectivity

Suppose $z \in E$ is orthogonal to $\text{im}(T)$. Choose a four-site representative v of z in $p^\perp$. It commutes with p and with every element of the graph, hence $v \in S^\perp$. Its weight on the original qubits is at most four.

Distance at least five forces $v \in S$. But $S \cap V_4 = \langle p \rangle$, so $z = 0$. Thus $(\text{im}(T))^\perp = 0$. Nondegeneracy of E gives $\text{im}(T) = E$, and therefore

$$\text{rank}(T) = 6, \quad \dim \ker(T) = 10 - 6 = 4.$$

Surjectivity also identifies the radical of the pairing on P :

$$\text{rad}(P) = P \cap P_W^\perp = \ker(T), \quad \dim \text{rad}(P) = 4.$$

Put

$$C = P_W^\perp.$$

Then $\dim C = 10$, so $|C| = 1024$. If a nonzero $c \in C$ had physical weight at most four, the suffix-only vector $(0, c)$ would be in $S^\perp$, and distance would force it into S . It cannot equal p , since its prefix is zero; it would therefore be another nonzero stabilizer word of weight at most four, contrary to the hypothesis. Thus $d(C) \geq 5$. Because $C^\perp = P$,

$$\dim(C \cap C_W^\perp) = \dim(P \cap C) = 4.$$

□

6.2 A necessary coset-capacity bound

Let $J = C \cap C_W^\perp$, so $\dim J = 4$, and let $P = C_W^\perp$. The quotient P/J is a nondegenerate six-dimensional symplectic space. Also $J_W^\perp/C$ has 64 suffix cosets. For $\xi = w + C \in J_W^\perp/C$, put

$$d(\xi) = \min_{c \in C} \text{wt}(w + c).$$

The pairing

$$(J_W^\perp/C) \times (P/J) \longrightarrow \mathbb{F}_2, \quad (w + C, x + J) \longmapsto \langle w, x \rangle_W$$

is well defined and nondegenerate: its left kernel is $P_W^\perp = C$, and both quotient spaces have dimension six.

For each $\xi = w + C$, there is a unique $e(\xi) \in E$ such that

$$\langle e(\xi), T(x) \rangle_E = \langle w, x \rangle_W \quad (x \in P).$$

The symplectic form on P/J identifies $(P/J)^*$ with P/J , and T induces a symplectic isomorphism $P/J \rightarrow E$. The perfect pairing above therefore identifies $J_W^\perp/C$ with E , so $\xi \mapsto e(\xi)$ is a bijection and sends zero to zero.

We use a counting argument to bound how many low-cost prefix classes a valid code can use. For $e \in E$, define its physical prefix cost by

$$\text{cost}(e) = \min\{\text{wt}(v) : v \in p^\perp \cap V_4, v + \langle p \rangle = e\}.$$

Its distribution is given in Table 5.

Table 5: Distribution of the physical prefix cost over the 64 classes of E .

cost	0	1	2	3	4
number of classes	1	4	27	24	8

To obtain these counts, split the even-weight x -masks in $p^\perp$ by weight. The $x = 0$ quotient contributes 1, 4, 3 classes of costs 0, 1, 2; each of the six weight-two x -masks contributes four classes of cost 2 and four of cost 3; the weight-four mask contributes eight classes of cost 4.

For every suffix representative $w \in \xi$, a prefix representative v of $e(\xi)$ makes $v + w$ commute with S . Conversely, commutation forces this prefix class for a given suffix class. Thus the mixed centralizer fiber indexed by ξ has minimum weight $d(\xi) + \text{cost}(e(\xi))$. If $\xi \neq 0$, its suffix is nonzero, so no vector in this fiber is p . Each vector in the fiber therefore has weight at least five: if it lies in S , Sections 4–5 leave p as the only nonzero stabilizer word of weight at most four; otherwise its weight is at least $d(S)$. Hence

$$d(\xi) + \text{cost}(e(\xi)) \geq 5 \quad (\xi \neq 0).$$

Every compatible graph map induces a symplectic isomorphism $P/J \rightarrow E$. The bijection $\xi \mapsto e(\xi)$ and the cost distribution give the following capacities without enumerating these isomorphisms:

$$\#\{\xi \neq 0 : d(\xi) \leq 1\} \leq 8,$$

$$\#\{\xi \neq 0 : d(\xi) \leq 2\} \leq 32,$$

$$\#\{\xi \neq 0 : d(\xi) \leq 3\} \leq 59.$$

The capacities are the numbers of nonzero prefix classes of cost at least 4, 3, and 2, respectively. Thus every symplectic lift must satisfy the three inequalities; failure of any one excludes it.

7 Exhaustive additive-code classification

7.1 What is classified

Under the standard Pauli/additive-code correspondence [1], represent a quaternary symbol by two binary bits. A length- n additive code is a binary subspace of $\mathbb{F}_2^{2n}$, with quaternary Hamming weight equal to physical Pauli weight. Equivalence is coordinate permutation plus an arbitrary local $\text{GL}(2, 2)$ map on each two-bit symbol plane. This is exactly the binary local symplectic monomial equivalence relevant here. No global $\text{GF}(4)$ scalar-linearity is assumed. Since $\text{GL}(2, 2) = \text{Sp}(2, 2)$, every such map preserves the symplectic form and Pauli weight. Hence the hull dimension of C and the minimum-weight spectrum of the cosets $J_W^\perp/C$ used in Section 6.2 are invariants of the equivalence class, so it suffices to check one representative per class.

Grassl, Krotov, Sok, and Solé [6] report 37 equivalence classes in the $(n, k) = (10, 10)$ cell of their additive $(n, 2^k, \geq 5)_4$ classification (Section 2.3, Table 1 of *The punctured dodecacode is unique*). They report the count, not the 37 generator matrices. We reconstruct the classes by lengthening. The published counts serve as cross-checks: the census generates candidates from predecessor cells and decides equivalence by the graph certificates described below; the table counts do not by themselves establish completeness.

7.2 Why the lengthening census is exhaustive for this cell

The recurrence starts from the zero code in the $(n, k) = (4, 0)$ cell and builds additive codes one coordinate at a time. The projection argument shows that every code in the target class arises from one of the extension cases below.

Let $D \leq \mathbb{F}_2^{2(m+1)}$ be a target code of binary dimension k , and project it onto its last two-bit coordinate. The image has binary dimension $r \in \{0, 1, 2\}$; the kernel is a length- m parent code of binary dimension $k - r$. Choose a basis of the image and lift its basis vectors to D . Their old-coordinate parts determine r quotient classes modulo the parent. For $r = 1$, the nonzero class must have minimum old weight at

least four. For $r = 2$, each of the three nonzero combinations of the two classes must have minimum old weight at least four. After a local $\text{GL}(2, 2)$ change of basis, these are exactly the extensions below. Conversely, each listed extension has distance at least five whenever its parent has distance at least five and the stated coset minima are at least four.

Thus lengthening a parent of length m and binary dimension $k - r$ uses one new coordinate of rank r :

- $r = 0$: append a zero coordinate;
- $r = 1$: choose one quotient coset of the parent whose old minimum weight is at least four and attach a nonzero new symbol;
- $r = 2$: choose two quotient labels whose three nonzero XOR combinations all have old minimum weight at least four and attach independent new symbols.

The threshold is four: a word with nonzero new symbol gains one physical unit of weight, while a word with zero new symbol lies in the distance-five parent. Thus every target code appears in one of these three cases.

Table 6 lists the predecessor cells needed to reach $(10, 10)$ and the reconstructed class counts:

Table 6: Predecessor cells required by the lengthening recurrence to reach $(10, 10)$, with reconstructed class counts.

length n	required binary dimensions k : number of classes
5	0 : 1, 1 : 1, 2 : 1, 3 : 0, 4 : 0
6	2 : 5, 3 : 1, 4 : 0, 5 : 0, 6 : 0
7	4 : 43, 5 : 1, 6 : 0, 7 : 0, 8 : 0
8	6 : 1579, 7 : 0, 8 : 0, 9 : 0, 10 : 0
9	8 : 2298, 9 : 0, 10 : 0
10	10 : 37

Every zero-count predecessor cell listed above is computed and verified empty rather than treated as an absent cell. Hence Table 6 covers every predecessor needed by the recurrence through $(10, 10)$.

7.3 Equivalence certificates

For every candidate the census constructs a colored incidence graph with one vertex for every codeword, three nonzero-symbol vertices for every coordinate, and one coordinate-group vertex binding each triple of symbols. The colors separate the three vertex types. A color-preserving graph isomorphism is exactly a reordering of codewords, a coordinate permutation, and an arbitrary permutation of the three nonzero symbols in each coordinate. The latter is $\text{GL}(2, 2)$. Thus the graph certificate gives a canonical additive-monomial equivalence test; the weight and pair-rank invariants only filter candidates before canonicalization. The graph-isomorphism implementation is based on nauty [7], via `pynauty` 2.8.8.1.

Completeness does not rest on the published counts. The census keeps one representative for every certificate it generates, so no orbit reached by an extension is discarded. As an internal check that no orbit was lost or duplicated, a second census implementation (`verification/independent/`) with a different graph encoding verifies, in every one of the 36 cells it computes, the orbit-stabilizer identity

$$\sum_P \frac{m! 6^m}{|\text{Aut } P|} \text{ext}_r(P) = \sum_D \frac{(m+1)! 6^{m+1}}{|\text{Aut } D|},$$

where P runs over parent classes, $\text{ext}_r(P)$ counts labelled extensions of rank r , and D runs over the classes found. In the target cell both sides equal 2,333,060,795,842,560. The automorphism-group orders of the 37 classes are listed in Table 7; the two censuses produce the same 37 canonical certificates.

Table 7: Hull dimension and automorphism-group order $|\text{Aut}|$ (coordinate permutations and per-site $\text{GL}(2, 2)$) of the 37 length-ten classes, numbered as in the repository.

class	hull	$ \text{Aut} $	class	hull	$ \text{Aut} $	class	hull	$ \text{Aut} $	class	hull	$ \text{Aut} $
1	2	2	2	2	1	3	0	2	4	0	10
5	2	2	6	0	2	7	0	2	8	0	30
9	4	48	10	2	30	11	2	24	12	4	16
13	2	2	14	4	12	15	4	6	16	0	2
17	4	16	18	2	8	19	2	1	20	4	5
21	4	16	22	4	1	23	2	2	24	8	6
25	0	2	26	4	5	27	2	2	28	2	5
29	6	5	30	6	10	31	2	5	32	8	27
33	2	8	34	4	8	35	4	16	36	2	10
37	4	8									

7.4 Hull and coset-capacity results

The census produces 37 generator matrices, each defining 1,024 words of minimum distance five. Separate audit scripts recompute their ranks, distances, hull dimensions, and coset minima. The hull dimensions are

Table 8: Binary symplectic hull dimensions of the 37 length-ten classes.

binary symplectic hull dimension	number of classes
0	7
2	14
4	12
6	2
8	2

By Table 8, the 25 classes with hull dimension different from four cannot arise from the code C required by Lemma 4. For the 12 hull-four classes, the coset-capacity audit gives Table 9:

Table 9: Coset-capacity audit of the 12 hull-four classes. Every class exceeds the capacity $N_{\leq 3} \leq 59$ of Section 6.2.

class	suffix weight spectrum $\{w : N_w\}$	$N_{\leq 1}$	$N_{\leq 2}$	$N_{\leq 3}$	capacity
9	$\{0 : 1, 2 : 39, 3 : 24\}$	0	39	63	fails
12	$\{0 : 1, 1 : 2, 2 : 37, 3 : 24\}$	2	39	63	fails
14	$\{0 : 1, 2 : 39, 3 : 24\}$	0	39	63	fails
15	$\{0 : 1, 2 : 27, 3 : 36\}$	0	27	63	fails
17	$\{0 : 1, 1 : 2, 2 : 37, 3 : 24\}$	2	39	63	fails
20	$\{0 : 1, 2 : 35, 3 : 27, 4 : 1\}$	0	35	62	fails
21	$\{0 : 1, 1 : 2, 2 : 37, 3 : 24\}$	2	39	63	fails
22	$\{0 : 1, 2 : 23, 3 : 40\}$	0	23	63	fails
26	$\{0 : 1, 2 : 35, 3 : 27, 4 : 1\}$	0	35	62	fails
34	$\{0 : 1, 1 : 2, 2 : 37, 3 : 24\}$	2	39	63	fails
35	$\{0 : 1, 1 : 2, 2 : 37, 3 : 24\}$	2	39	63	fails
37	$\{0 : 1, 1 : 2, 2 : 37, 3 : 24\}$	2	39	63	fails

In every row $N_{\leq 3} \in \{62, 63\} > 59$. Eight rows also have $N_{\leq 2} = 39 > 32$.

Lemma 5 (Additive lift exclusion). *None of the 37 length-ten classes can supply the code in Lemma 4: 25 have hull dimension different from four, and the remaining 12 violate the coset-capacity bound.*

Proof. The first exclusion follows from Table 8. For every remaining class, Table 9 gives $N_{\leq 3} \in \{62, 63\}$, exceeding the necessary capacity 59. Hence none of the 37 classes can supply the required code. $\square$

8 Discussion and conclusion

Proof of Theorem 1. Assume a binary $[[14, 3, d \geq 5]]$ stabilizer code exists. Sections 3–4 give $A_1 = A_3 = 0$, $A_2 \leq 2$, and $A_4 \in \{1, 3\}$. Section 5.3 then excludes weight-two stabilizer words, so $A_2 = 0$. Sections 5.3–5.5 exclude all pairs of distinct weight-four stabilizer words whose supports intersect in at least two sites and classify the $A_4 = 3$ case into five support geometries. Section 5.6 excludes the four overlapping rank-three geometries, and Section 5.7 excludes the disjoint $4 + 4 + 4 + 2$ geometry. Hence $A_4 = 1$, which is Lemma 3.

Lemma 4 then forces a ten-site additive code of size 1,024, distance at least five, and hull dimension four. By Lemma 5, none of the 37 equivalence classes of length-ten additive codes of size 1,024 and distance at least five can supply the code required by Lemma 4: 25 have the wrong hull dimension, and the remaining 12 violate the coset-capacity bound. This is a contradiction. $\square$

The proof applies to these parameters and to stabilizer codes; it gives no bound at other lengths or for non-stabilizer codes.

9 Computational material

The verification uses exact integer Farkas certificates for the split-weight systems and an exhaustive classification of length-ten additive codes. No floating-point computation is used as a proof step. The repository is

<https://github.com/brandonlign/quantum-code-bounds>

With Python 3.10 or newer and Node.js, the command

```
bash verification/replay.sh
```

replays every certificate of Sections 4–7, most of them in two separate implementations (Python and JavaScript with BigInt), and checks the 37 committed class representatives. The option `--census` also regenerates the classification from scratch, which takes a few minutes and requires `pynauty==2.8.8.1`. The script names follow the section numbers of this paper.

A second implementation in `verification/independent/`, written separately from the first from the manuscript text, rebuilds the argument: the Lemma 2 identity and parity congruence, the completeness of Table 1, all Section 5 relaxations (with a floating-point LP solver, as corroboration only), an independent census with the orbit-stabilizer check of Section 7.3, and the hull and coset spectra of Table 9. Both implementations were written by the author with the assistance described in the acknowledgments. They are cross-checks of one another, not verification by a separate party.

For readers who prefer to check Section 5 without reading either implementation, every Farkas certificate is published as a self-describing data file in `verification/certificates/`, in the row-by-row format

of Appendix A. The short standalone checker `verification/check_certificates.py` rebuilds each system from that specification alone. Writing a different checker from Appendix A is intended to be a modest exercise.

Acknowledgments

Generative-AI tools (ChatGPT, Codex and Claude) assisted with exploring proof strategies, checking derivations, writing verification code, and editing the text. The author directed the research, checked the mathematical arguments and computational results, and takes full responsibility for the content. The author thanks Simeon Ball for comments on an earlier version of the manuscript. This acknowledgment does not imply that he has checked the proof.

Statements and declarations

Data availability. All code, certificate data and the 37 class representatives are publicly available in the repository given in Section 9. They are also archived on Zenodo at [doi:10.5281/zenodo.22885774](https://doi.org/10.5281/zenodo.22885774).

Funding. No funding was received for this work.

Competing interests. The author has no competing interests to declare.

References

- [1] A. R. Calderbank, E. M. Rains, P. W. Shor, and N. J. A. Sloane, “Quantum error correction via codes over $GF(4)$,” *IEEE Trans. Inf. Theory* 44 (1998), 1369–1387. [doi:10.1109/18.681315](https://doi.org/10.1109/18.681315).
- [2] E. M. Rains, “Quantum weight enumerators,” *IEEE Trans. Inf. Theory* 44 (1998), 1388–1394. [doi:10.1109/18.681316](https://doi.org/10.1109/18.681316).
- [3] E. M. Rains, “Quantum shadow enumerators,” *IEEE Trans. Inf. Theory* 45 (1999), 2361–2366. [doi:10.1109/18.796376](https://doi.org/10.1109/18.796376).
- [4] S. Ball, A. Centelles, and F. Huber, “Quantum error-correcting codes and their geometries,” *Ann. Inst. H. Poincaré D* 10 (2023), no. 2, 337–405. [doi:10.4171/AIHPD/160](https://doi.org/10.4171/AIHPD/160).
- [5] J. Bierbrauer, R. Fears, S. Marcugini, and F. Pambianco, “The nonexistence of a $[[13,5,4]]$ -quantum stabilizer code,” *IEEE Trans. Inf. Theory* 57 (2011), 4788–4793. [doi:10.1109/TIT.2011.2146430](https://doi.org/10.1109/TIT.2011.2146430).
- [6] M. Grassl, D. Krotov, L. Sok, and P. Solé, “The punctured dodecacode is unique,” *Mathematics and Education in Mathematics* 55 (2026), 393–404. [doi:10.55630/mem.2026.55.393-404](https://doi.org/10.55630/mem.2026.55.393-404).
- [7] B. D. McKay and A. Piperno, “Practical graph isomorphism, II,” *J. Symbolic Comput.* 60 (2014), 94–112. [doi:10.1016/j.jsc.2013.09.003](https://doi.org/10.1016/j.jsc.2013.09.003).
- [8] M. Grassl, “Bounds on the minimum distance of additive quantum codes, (n=14,k=3,q=4),” maintained code table, accessed 24 September 2026. codetables.de.

A Exact specification of the Section 5 systems

This appendix defines every linear system of Section 5 in enough detail that a reader can rebuild it, and check its certificate, without reference to any code supplied by the author.

A.1 Variables

The fourteen qubits are split into consecutive blocks of sizes $m_1, \dots, m_r$ and a suffix of size f , as listed in Table 11. For a block i with subgroup $R_i \leq V_{m_i}$, compute the normalizer $R_i^\perp \cap V_{m_i}$ and split it into complete R_i -cosets. Each coset Q has a weight histogram $p(Q) = (p_0, \dots, p_{m_i})$ as in Section 5.1. The distinct histograms, sorted lexicographically, are the *patterns* $P_{i,1}, P_{i,2}, \dots$ of block i .

A column is a tuple $y = (k_1, \dots, k_r, b)$: one pattern index for each block and a suffix weight $0 \leq b \leq f$. Columns are ordered lexicographically. Its variable counts the R -cosets of S (where $R = R_1 \oplus \dots \oplus R_r$) whose block histograms are $P_{1,k_1}, \dots, P_{r,k_r}$ and whose suffix part has weight b . In the two systems marked “cells” in Table 11, the variables are instead the split counts $H_{a,b}$ themselves, which is the same as taking the patterns of block 1 to be the unit vectors $e_0, \dots, e_{m_1}$.

A split cell is $c = (a_1, \dots, a_r, a_f)$ with $0 \leq a_i \leq m_i$, $0 \leq a_f \leq f$, and $|c| = \sum a_i + a_f$. The contribution of column y to the three enumerators in cell c is

$$\begin{aligned} H_c(y) &= \prod_{i=1}^r P_{i,k_i}[a_i] \cdot [a_f = b], \\ T_c(y) &= \prod_{i=1}^r \left(\sum_j K_{a_i}^{m_i}(j) P_{i,k_i}[j] \right) \cdot K_{a_f}^f(b), \\ \text{SH}_c(y) &= \prod_{i=1}^r \left(\sum_j (-1)^j K_{a_i}^{m_i}(j) P_{i,k_i}[j] \right) \cdot (-1)^b K_{a_f}^f(b), \end{aligned}$$

which is the tensor-product form of the transforms in Section 3. Here $T_c = 2048 C_c$ and $\text{SH}_c = 2048 \text{Sh}_c$.

A.2 Rows

Each constraint is identified by a type and, where relevant, a cell. Table 10 gives the coefficient of column y in each type. Every row is a necessary condition on a hypothetical code containing R , by Sections 3–5.1.

Table 10: Row types. Equalities (=) take free-sign multipliers ν ; inequalities ($\leq$) take multipliers $\lambda \geq 0$.

type	coefficient of column y	relation
unit	$H_{(0,\dots,0)}(y)$	$= 1$
total	$\sum_c H_c(y)$	$= 2048$
low c, $1 \leq c \leq 4$	$T_c(y) - 2048 H_c(y)$	$= 0$
zero c, $c \in \{1, 3\}$	$H_c(y)$	$= 0$
weight w	$\sum_{ c =w} H_c(y)$	$= A_w$
coset_eq, coset_le	$\sum_a \omega_a H_{(a,b)}(y)$ for given ω, b	$= 0, \leq 0$
incl c	$2048 H_c(y) - T_c(y)$	≤ 0
cnonneg c	$-T_c(y)$	≤ 0
shadow c	$-\text{SH}_c(y)$	≤ 0

The **weight** rows use only the values $A_1 = A_2 = A_3 = 0$ and the branch assumption $A_4 = 3$, and appear only in Sections 5.6–5.7. The **zero** row uses $A_1 = 0$ from Lemma 2 and appears only in Section 5.4.2. The **coset** rows appear only in the two “cells” systems, where they encode the coset relations stated in Sections 5.3 and 5.4.2: $H_{2,b} \geq H_{0,b}$, $H_{1,b} = 0$, and $H_{4,b} = H_{2,b} + 3H_{0,b}$. Each such relation is implied by every coset pattern of its subgroup, which a checker should confirm.

A.3 Certificates

A certificate is a finite list of (row, multiplier) pairs. For each system, the residual $c(y) = \sum \lambda_\rho G_\rho(y) + \sum \nu_\rho M_\rho(y)$ must satisfy $c(y) \geq 0$ in every column, with $\nu \cdot \text{rhs} < 0$. Section 5.2 shows that no code satisfies the system in that case. Table 11 lists all eleven systems. The multipliers themselves are in `verification/certificates/`, one JSON file per row of the table. Each file contains the block sizes, the subgroup generators, the coset patterns with multiplicities, and the labelled rows with their integer multipliers. The certificates are sparse: they have between 22 and 111 nonzero multipliers.

Table 11: The eleven Section 5 systems. Blocks give $(m_1, \dots, m_r; f)$. “Pat.” is the number of coset patterns per block. Generators act on the block sites in order, and in Section 5.7 each of the three blocks carries ZZZZ.

§	generators of R	blocks	pat.	columns	rows used	$\nu \cdot \text{rhs}$
5.3	ZZ	(2; 12)	cells	39	23	−2571108352
5.4.1	ZZZ	(3; 11)	3	36	22	−22528
5.4.2	XXXX, ZZZZ	(4; 10)	cells	55	39	−320017816092672
5.4.3	XXZZI, ZZZIZ	(5; 9)	9	90	38	−52501014528
5.4.4	ZZZZII, ZZIIZZ	(6; 8)	13	117	25	−8820736
5.4.4	ZZZZII, XXIIXX	(6; 8)	14	126	27	−22212608
5.6	three distinct intersections	(9; 5)	40	240	32	−86470656
5.6	one common site	(10; 4)	27	135	33	−9031680
5.6	two intersections	(10; 4)	60	300	34	−20484096
5.6	one intersection	(11; 3)	74	296	33	−61440
5.7	ZZZZ on each block	(4, 4, 4; 2)	6,6,6	648	111	−125829120

The Section 5.6 generators are the Z-only words on the supports in Table 2. The patterns of the 5.4.1, 5.4.2 and 5.7 blocks appear in Sections 5.4.1, 5.4.2 and Table 4. For the remaining two-generator blocks, the patterns (written $(p_0, \dots, p_m)$:multiplicity) are:

- $\langle \text{XXZZI}, \text{ZZZIZ} \rangle$: $(0, 0, 0, 0, 4, 0):12, (0, 0, 0, 1, 0, 3):8, (0, 0, 0, 2, 0, 2):12, (0, 0, 0, 3, 0, 1):12, (0, 0, 0, 4, 0, 0):1, (0, 0, 1, 0, 3, 0):12, (0, 0, 2, 0, 2, 0):3, (0, 1, 0, 2, 0, 1):3, (1, 0, 0, 0, 3, 0):1$.
- $\langle \text{ZZZZII}, \text{ZZIIZZ} \rangle$: $(0, 0, 0, 0, 0, 4):16, (0, 0, 0, 0, 0, 4, 0):24, (0, 0, 0, 0, 2, 0, 2):24, (0, 0, 0, 0, 3, 0, 1):64, (0, 0, 0, 0, 4, 0, 0):12, (0, 0, 0, 1, 0, 3, 0):64, (0, 0, 0, 2, 0, 2, 0):24, (0, 0, 0, 4, 0, 0, 0):2, (0, 0, 1, 0, 2, 0, 1):12, (0, 0, 2, 0, 2, 0, 0):6, (0, 0, 3, 0, 0, 0, 1):1, (0, 1, 0, 2, 0, 1, 0):6, (1, 0, 0, 0, 3, 0, 0):1$.
- $\langle \text{ZZZZII}, \text{XXIIXX} \rangle$: $(0, 0, 0, 0, 0, 4, 0):32, (0, 0, 0, 0, 1, 0, 3):16, (0, 0, 0, 0, 2, 0, 2):32, (0, 0, 0, 0, 3, 0, 1):40, (0, 0, 0, 0, 4, 0, 0):16, (0, 0, 0, 1, 0, 3, 0):48, (0, 0, 0, 2, 0, 2, 0):32, (0, 0, 0, 3, 0, 1, 0):4, (0, 0, 1, 0, 1, 0, 2):8, (0, 0, 1, 0, 2, 0, 1):16, (0, 0, 1, 0, 3, 0, 0):5, (0, 0, 2, 0, 1, 0, 1):2, (0, 1, 0, 1, 0, 2, 0):4, (1, 0, 0, 0, 2, 0, 1):1$.

The 40–74 patterns of the Section 5.6 blocks are too long to print, but they are determined by the generators. They are listed in the certificate files, and `check_certificates.py` recomputes them before using them.